PIPELINE

Figure 2 | R97 端到端工作流 (End-to-End Workflow Pipeline)

对标 MINIAUTH (ACM CCS) 宽屏分层工作流范式 · 从混淆输入到保语义代码生成的全流程流水线

阶段 I: 语法解析与特征提取 (Ingestion)

1. 多源混淆代码包输入

- 微信小程序包 (app-service.js / 1572项目)
- 前端 Web 平台测试包 (WPT / 900文件)
- 后端 Node.js 服务端包 (Node / 900文件)

➜ 生成解析用例流与原始 AST 节点

2. AST 作用域与词法遮蔽分析器

- 词法作用域树构建：自顶向下解析作用域链
- 形参遮蔽过滤：function(wx) 消除全局伪依赖
- 常量别名链追踪：const client = wx 追溯
- 宿主归属表匹配：区分宿主专属根与语言内建

➜ 产出待折叠候选集合 (Candidate Findings)

3. 前置极简快速分流 (Pre-Filter)

分支 A: 纯局部变量 / 纯内建数值计算

不依赖宿主能力 ➜ 快速通道直接放行 (Fast-Path)

分支 B: 包含外部宿主 API 访问或动态键

进入 R97 核心守门流水线 (Guard Pipeline) ➜

阶段 II: 证据确立与隔离审查

1. 外部部署证据解析 (Target Resolver)

- 摄取 project.config.json / package.json
- 提取平台标识、基础库版本、构建指纹
- 证据定级：confirmed (强证实) vs inferred

➜ 输出客观目标环境: target = { Wechat, confirmed }

未确认目标一律禁止授权折叠！

🔒 **正交隔离审查防火墙**

Required ⊥ Target (禁止循环论证)

- 代码有 wx 仅代表依赖需求 (required)
- 严禁借此反推目标环境！目标必须由外部独立给出

➜ 审查通过后构造 DecisionQuery 进入判卷

3. 判定查询构造 (DecisionQuery)

- 将每个候选点绑定为五元组：

  `Query = (pointId, entity, target, evalHost, version)`

- 为每个点定制唯一的 SHA-256 唯一防伪 ID

➜ 并行分发给中央决策引擎与探针比对 ➜

阶段 III: 契约核验与探针实测 (Probing)

1. 语义契约投影与非确定性陷阱核验

- 投影 AST 运算维度：[typeof, truthiness, return_val]
- 拦截 Date.now / Math.random 等时变调用 (OBsmith §3.1)

➜ 契约约束集生成：C = (strict_equal, requiredDims)

2. 4 宿主多执行面实测比对 (JEST 拓扑对标)

① 隔离 VM	② 原生 Node	③ Edge 浏览器	④ 微信 DevTools
纯语言基线	553 条实测差分	780 条实测差分	785 条微信差分

实测差异案例：`typeof wx`

VM/Node/Edge: 'undefined' ≠ 微信真机: 'object'

➜ 检测到跨宿主执行分歧 (Divergence Detected)!

- 证据库持久化：SHA-256 签名 + SDK版本时效失效机制
- **Level-2 动态执行验证：386 样本行为不一致率 0.00%**

3. 八步安全判定核心 D(req, target, C, E)

①绑定 ➜ ②契约 ➜ ③明确性 ➜ ④目标证实 ➜ ⑤硬缺 ➜ ⑥证据 ➜ ⑦投影 ➜ ⑧归约

- **严格等价且可折叠 ➜ 发放 FOLD 凭证**
- **实测存在环境冲突 ➜ 输出 PROTECT 强力拦截**
- **目标未确认或证据不足 ➜ 安全兜底 UNKNOWN**

工作流总结：前端 WPT / 后端 Node / 微信小程序源码 ➜ AST 解析与外部指纹解耦 ➜ 契约与 4 宿主实测探针核验 ➜ 三值网关拦截冲突 ➜ 保语义安全反混淆输出。